

Subgraphs in LangGraph — Study Notes

Notes on what subgraphs are, why they are needed for complex GenAI/agent workflows, and how to implement them in LangGraph, with a worked translation-workflow example.

1 What Is a Subgraph?

Any AI workflow can be represented as a graph, where each **node** represents a task — calling an LLM, retrieving from a vector database, invoking a tool, and so on.

Definition: A subgraph is a graph that is embedded and executed as a node inside another (parent) graph. In other words, instead of a node performing a single task directly, that node is replaced by an entire graph of its own.

If a node in a larger graph is itself a complete graph, that inner graph is called a **subgraph**, and the outer graph is the **parent graph**.

2 Why Subgraphs Are Needed

Early GenAI applications tend to be simple: a user query goes to an LLM, and the LLM returns an output. As requirements grow, real-world GenAI systems become complex — they may need tool calling, retrieval (RAG), conditional routing, retry logic, memory, human-in-the-loop steps, evaluation, and guardrails.

2.1 Motivating Example: A Software Development Agent

Consider building an agent that mimics a software development process:

- A team lead plans and assigns work.
- A backend team and a frontend team build the software.
- The output is tested.
- The code is reviewed.
- A DevOps team deploys and monitors the system.

Implementing this as one single, flat LangGraph graph would be extremely complex, since every stage might need its own tools, retry logic, memory, and guardrails.

Solution: Break the large agent into smaller agents — a planning agent, two coding agents (back-end/frontend), a testing agent, a code-review agent, and a DevOps agent — forming a **multi-agent architecture**. Each individual agent is represented as a subgraph, with its own internal logic, tools, memory, evaluation, and guardrails fully self-contained.

3 Core Benefits of Subgraphs

3.1 General (Conceptual) Benefits

1. **Modularity** — the overall system is broken down into self-contained functional units, following the general software principle that code should be modular.
2. **Reusability** — a subgraph built for one purpose (e.g., a coding agent) can be reused elsewhere (e.g., for both frontend and backend teams), since the underlying logic is essentially the same.
3. **Maintainability** — debugging becomes easier, since a problem can be traced directly to the specific subgraph responsible, rather than searching through one massive graph.

3.2 LangGraph-Specific Benefits

1. **Failure Isolation** — if one subgraph fails or errors out, the rest of the parent graph can still execute (with warnings), rather than bringing down the entire workflow.
2. **State Separation** — each subgraph can maintain its own independent state rather than forcing every component to share a single, large, tangled state object.
3. **Observability** — tools such as LangSmith can trace subgraphs individually and at a granular level, making it possible to monitor token usage, latency, and behavior for a specific agent (e.g., the coding agent) in isolation.

4 Two Mechanisms for Implementing Subgraphs

LangGraph documentation specifies that when adding a subgraph, you must define how the parent graph and the subgraph communicate. There are two mechanisms:

Mechanism 1 — Invoke a graph from within a node.

The subgraph is built completely independently, with its own separate state. Inside a specific node of the parent graph, the subgraph is simply invoked (called), and its output is extracted and written back into the parent's state.

Mechanism 2 — Add a graph directly as a node.

Rather than writing a node function, the compiled subgraph itself is added as a node in the parent graph. In this mode, the subgraph shares state keys directly with the parent — there is a single shared state rather than two separate ones.

The key difference: in Mechanism 1, the parent and subgraph each have their own separate state; in Mechanism 2, the subgraph operates on the same state object as the parent.

5 Worked Example: English-to-Hindi Translation Workflow

Use case: A user asks a question. An LLM generates an English answer. Since the user wants the answer in Hindi, a second LLM call translates the English answer into Hindi.

5.1 Mechanism 1: Isolated States

Subgraph (translation graph):

- **State (SubState):** `input_text` (English answer), `translated_text` (Hindi answer).
- **Single node** — calls an LLM with a prompt instructing it to translate the given text into natural, clear Hindi without adding extra content, and stores the result in `translated_text`.
- Graph: `START` → `translate` → `END`, compiled independently.

Parent graph:

- **State (ParentState):** `question`, `english_answer`, `hindi_answer` (all strings).
- **Node 1 — generate:** invokes the parent LLM with a system instruction to act as a helpful assistant and answer the question; result stored as `english_answer`.
- **Node 2 — translate:** invokes the compiled subgraph, passing in `english_answer`. The subgraph returns its full final state; only `translated_text` is extracted and written into `ParentState.hindi_answer`.
- Graph: `START` → `generate` → `translate` → `END`, compiled and invoked with the initial question.

Running the parent graph returns a final state containing the original question, the English answer, and the Hindi answer.

5.2 Mechanism 2: Shared State

The same use case can be implemented with a single shared state instead of two separate ones:

- Only **one state** is defined for the parent (`question`, `english_answer`, `hindi_answer`) — no separate substate.
- Two LLMs are still defined (one for generation, one for translation).
- The subgraph (translation logic, single node) is built exactly as before and compiled independently.
- The parent graph now defines only **one** explicit node — `generate`. The second "node" is the compiled subgraph itself, added directly as a node (`START` → `generate` → `subgraph` → `END`).
- Because the subgraph shares state keys with the parent, no manual extraction step is needed — the subgraph reads and writes directly into the shared state.

Both mechanisms produce the same final answer; they differ only in whether state is isolated or shared between parent and subgraph.

6 Best Practices and Additional Capabilities

- **Persistence:** Attaching a checkpoint to the parent graph is usually sufficient — LangGraph automatically checkpoints the child subgraph as well.
- **Streaming:** Subgraph outputs can be streamed just like outputs from a regular graph.
- **Viewing subgraph state:** LangGraph provides ways to inspect the internal state of a subgraph directly, useful for debugging nested workflows.

7 Quick Revision Summary

1. A subgraph is a graph used as a node inside another (parent) graph.
2. Subgraphs are essential for building multi-agent systems, where each agent is represented as its own subgraph.

3. Core benefits: modularity, reusability, and maintainability (general), plus failure isolation, state separation, and observability (LangGraph-specific).
4. Two implementation mechanisms: (1) invoke a subgraph from inside a node with an isolated state, or (2) add a compiled subgraph directly as a node, sharing state with the parent.
5. Mechanism 1 requires manually extracting subgraph output into parent state; Mechanism 2 shares state keys automatically, avoiding that extra step.
6. LangGraph also supports checkpointing, streaming, and state inspection for subgraphs, enabling persistence and observability in nested, multi-agent workflows.